

Adapter

Definition

Real Life Examples

WeighingScale

Plug, Socket & Power Adapter

XMLJSONParser

Class Diagram

Structure of the Adapter Design Pattern

Implementation

▼ Resources

• Video → [32. All Structural Design Patterns | Decorator, Proxy, Composite, Adapter, Bridge, Facade, FlyWeight](#)

• Video → [20. Adapter Design Pattern with Examples, LLD | Low Level Design Interview Question | System Design](#)

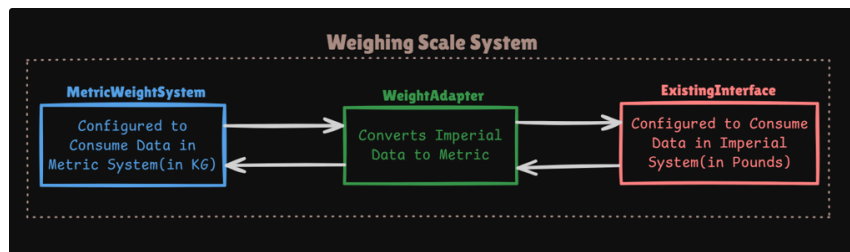
Definition



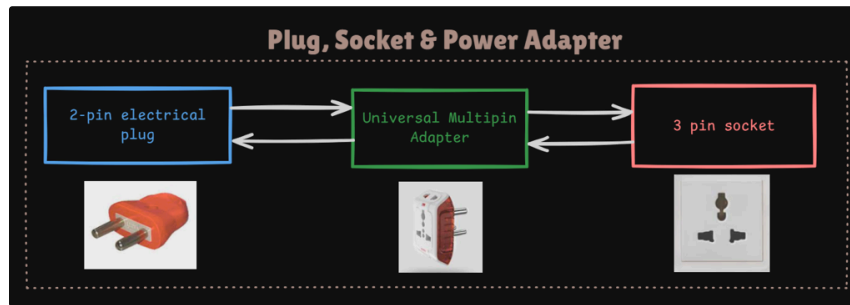
*The Adapter Pattern is a structural design pattern that serves as a **bridge** between two **incompatible interfaces**, enabling them to work together by providing data in a format the client expects.*

Real Life Examples

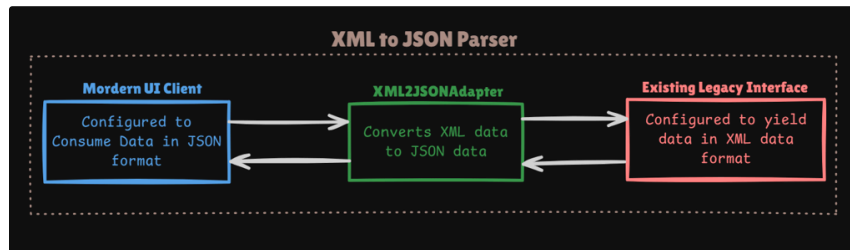
WeighingScale



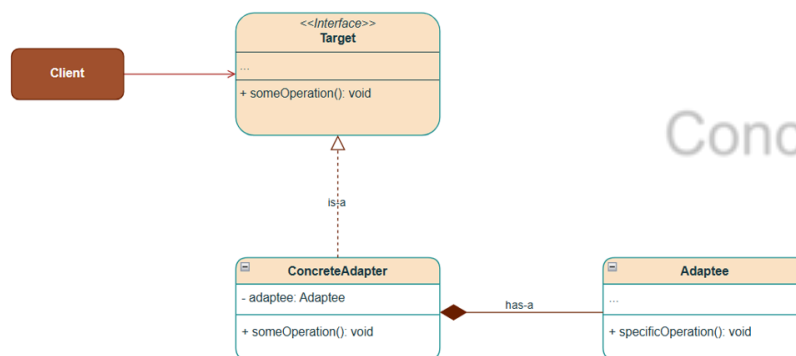
Plug, Socket & Power Adapter



XMLJSONParser

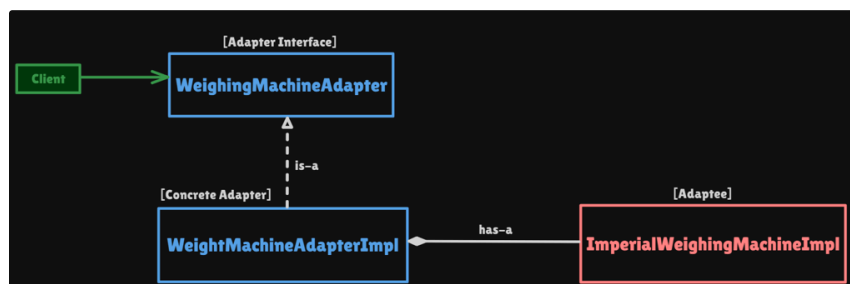


Class Diagram



Concept && Coding

Structure of the Adapter Design Pattern



1. **Target or Adapter Interface(WeighingMachineAdapter)**: This is what the client expects, e.g., a Weight Machine that measures weight in kg.
2. **Adaptee(ImperialWeighingMachineImpl)**: This is an existing incompatible class, e.g., that gives weight in pounds.

3. **Concrete Adapter(WeightMachineAdapterImpl)**: Adapts to the client's specific expectation and does the conversion/translation. It converts **pounds to kilograms**. Uses the Adaptee class internally for the translation/conversion.
4. **Client(MetricWeighingMachine)**: Utilizes the target interface without concern for conversion.

Implementation

Implementation of the Weighing Scale System using Adapter Design Pattern:

```
1 // Adaptee Interface
2 public interface ImperialWeighingMachine {
3     //return the weight in Pounds
4     double getWeightInPounds();
5 }
6
7 // Adaptee - Existing Incompatible class
8 public class ImperialWeighingMachineImpl implements
ImperialWeighingMachine {
9     double weightInPounds = 0;
10
11     public ImperialWeighingMachineImpl(double weighingScaleReading) {
12         this.weightInPounds = weighingScaleReading;
13     }
14
15     // Third-party weighing machine (US model) - returns pounds
16     @Override
17     public double getWeightInPounds() {
18         return weightInPounds;
19     }
20 }
```

```
1 // Target or Adapter Interface
2 public interface WeighingMachineAdapter {
3     double getWeightInKg(); // Client wants weight in KG
4 }
```

```
1 // Concrete Adapter converts pounds → kg
2 public class WeightMachineAdapterImpl implements
WeighingMachineAdapter {
3
4     // Adaptee Reference
5     ImperialWeighingMachine imperialWeighingMachine;
6
7     public WeightMachineAdapterImpl(ImperialWeighingMachine
weightMachineInPounds) {
8         this.imperialWeighingMachine = weightMachineInPounds;
9     }
10
11     @Override
12     public double getWeightInKg() {
13         double weightInPound =
imperialWeighingMachine.getWeightInPounds();
14         // Conversion formula: 1 pound = 0.453592 kg
15         return weightInPound * 0.45;
16     }
17 }
```

```
1 // Client - Metric Weighing Machine
2 public class MetricWeighingMachine {
3     public static void main(String[] args) {
4         System.out.println("===== Adapter Design Pattern =====");
5
6         // ImperialWeighingMachine - // Existing weighing machine is
used to weigh the baby in pounds
7         double weighingScaleReading = 25.0; // say the baby's weight
is 25 pounds
```

```
8      ImperialWeighingMachineImpl imperialWeighingMachine = new
ImperialWeighingMachineImpl(weighingScaleReading);
9
10     // Adapter to convert to KG
11     WeighingMachineAdapter weightMachineAdapter = new
WeightMachineAdapterImpl(imperialWeighingMachine);
12
13     // Client gets weight in Kilograms
14     System.out.println("Weight in KG: " +
weightMachineAdapter.getWeightInKg());
15 }
16 }
```

Concept && Coding

Bridge

Definition

The Problem

Solution: Bridge Design Pattern

Class Diagram

Structure of the Bridge Pattern

Implementation

Bridge Pattern vs Strategy Pattern

Resources

- Video → [32. All Structural Design Patterns | Decorator, Proxy, Composite, Adapter, Bridge, Facade, FlyWeight](#)
- Video → [26. Bridge Design Pattern | LLD of Bridge Pattern with Example | Low Level Design of Bridge Pattern](#)

Definition

The Bridge Design Pattern *decouples* an *abstraction* (high-level logic defines the "what") from its *implementation* (low-level details that define the "how") so that the two can *evolve independently*.

Concept && Coding

The Problem

Let's consider an example of different living things with different respiratory mechanisms.

The naive approach of this implementation would be to introduce an abstraction for the respiratory mechanism, i.e. breathing process, and extend it to define the breathing process for each new type of living thing we introduce.

Let's look at the code example:

```
1 // Approach without bridge: breathing logic hardcoded inside
  LivingThings
2 public abstract class LivingThings {
3     abstract public void breathe();
4 }

1 public class Dog extends LivingThings {
2
3     // Breathing Process is tightly coupled to the
    LivingThings(abstraction)
4     @Override
5     public void breathe() {
6         System.out.println("Dog: Breathes through its nose; Lives on
        land; Respiratory system: 2 lungs");
7         System.out.println("Breathing Process: Inhales Oxygen from the
        air and Exhales Carbon Dioxide.");
8     }
9 }

1 public class Fish extends LivingThings {
2
3     // Breathing Process is tightly coupled to the
    LivingThings(abstraction)
4     @Override
```

```

5     public void breathe() {
6         System.out.println("Fish: Breathes through gills; Lives in
water; Respiratory system: 2 gills");
7         System.out.println("Breathing Process: Absorbs Oxygen from the
water and releases Carbon Dioxide.");
8     }
9 }

```

```

1 public class Tree extends LivingThings {
2
3     // Breathing Process is tightly coupled to the
LivingThings(abstraction)
4     @Override
5     public void breathe() {
6         System.out.println("Tree: Breathes through leaves; Lives on
land; Respiratory system: Leaves");
7         System.out.println("Breathing Process: Inhales Carbon Dioxide
and Exhales Oxygen as a result of photosynthesis.");
8     }
9 }

```

```

1 public class Whale extends LivingThings {
2
3     // Breathing Process is tightly coupled to the
LivingThings(abstraction)
4     @Override
5     public void breathe() {
6         System.out.println("Whale: Breathes through lungs; Lives in
water; Respiratory system: 2 lungs");
7         System.out.println("Breathing Process: Inhales Oxygen from the
water and Exhales Carbon Dioxide");
8     }
9 }

```

```

1 // Client
2 public class Demo {
3     public static void main(String[] args) {
4         System.out.println("==== Bridge Design Pattern - Problem
Demo =====");
5         LivingThings dog = new Dog();
6         dog.breathe();
7         LivingThings fish = new Fish();
8         fish.breathe();
9         LivingThings whale = new Whale();
10        whale.breathe();
11        LivingThings tree = new Tree();
12        tree.breathe();
13    }
14 }

```

Concept & Coding

Problems Here:

1. Code Duplication

- Both **Dog** and **Whale** breathe through lungs → same logic repeated in multiple classes.

2. Tight Coupling

- Breathing logic is tied to each animal class. You can't easily **reuse** breathing behavior for another animal or introduce a new type independently.

3. Class Explosion

- Without Bridge, you might end up making separate classes for every combination:

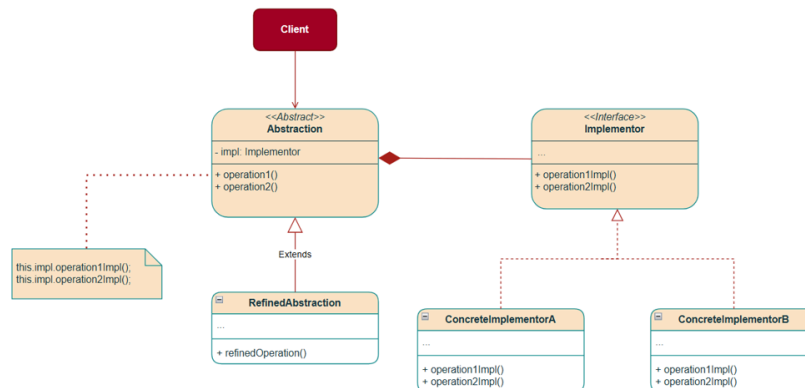
DogWithLungs, **WhaleWithLungs**, **FishWithGills**, etc.

Solution: Bridge Design Pattern

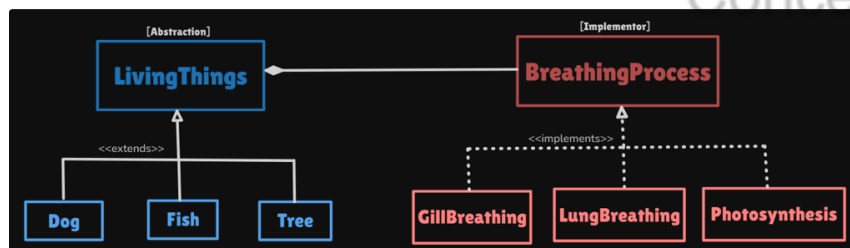
With the Bridge Pattern,

- Living Things (Dog, Fish, Whale, Tree) don't care how breathing is done.
- Breathing strategies (LungBreathing, GillBreathing, SkinBreathing) are separate.
- You can mix & match at runtime.
- Add a new breathing type? Just implement the Breathing Process.
- Add a new animal? Just extend Animal and plug in a breathing process.

Class Diagram



Structure of the Bridge Pattern



- **Abstraction** (**LivingThings**): High-level concept of a living thing and its operations.
- **Refined Abstraction** (**Dog** , **Fish** , **Whale** , **Tree**): Concrete types of living things with more refined functions.
- **Implementor** (**BreatheProcess**): Defines "how" breathing happens, the low-level details.
- **Concrete Implementors** (**LungBreathing** , **GillBreathing**): Actual breathing mechanisms defining reusable methods for respiratory mechanisms.

Implementation

```
1 //Step 1: Implementor (Breathing process)
2 public interface BreathingProcess {
3     void breathe();
4 }
```

```
1 // Step 2: Concrete Implementor (various breathing processes)
2 public class GillBreathing implements BreathingProcess {
3     @Override
4     public void breathe() {
```

```

5         System.out.println("Breathing through gills.");
6     }
7 }
8 public class LungBreathing implements BreathingProcess {
9     @Override
10    public void breathe() {
11        System.out.println("Breathing through lungs.");
12    }
13 }
14 public class Photosynthesis implements BreathingProcess {
15     @Override
16    public void breathe() {
17        System.out.println("Breathing through process of
18    photosynthesis. Releases Oxygen through leaves.");
19    }
20 }

```

```

1 //Step 3: Abstraction for LivingThings
2 public abstract class LivingThings {
3     // Reference to Implementor
4     protected BreathingProcess breathingProcess;
5
6     // Bridge connects Animal with Breathing
7     public LivingThings(BreathingProcess breathingProcess) {
8         this.breathingProcess = breathingProcess;
9     }
10
11    // Operation implemented by Implementor
12    abstract public void breathe();
13 }

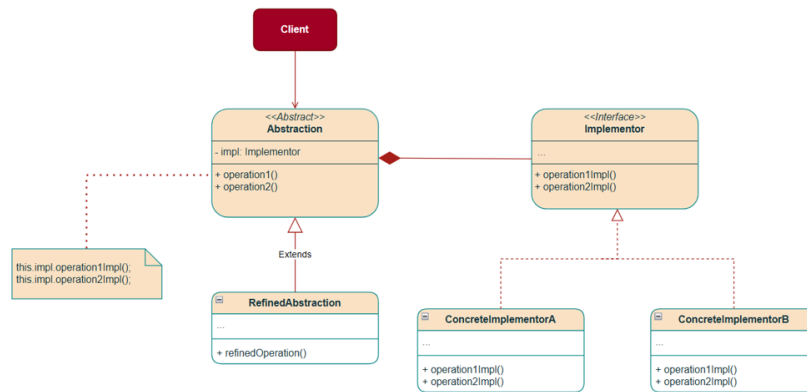
```

```

1 //Step 4: Refined Abstractions (Concrete LivingThings)
2 public class Dog extends LivingThings {
3
4     public Dog(BreathingProcess breathingProcess) {
5         super(breathingProcess);
6     }
7
8     @Override
9     public void breathe() {
10        System.out.print("Dog: ");
11        breathingProcess.breathe(); // Operation implemented by
12        Implementor - defines the "HOW"
13    }
14 }
15 public class Fish extends LivingThings {
16     public Fish(BreathingProcess breathingProcess) {
17         super(breathingProcess);
18     }
19
20     @Override
21     public void breathe() {
22        System.out.print("Fish: ");
23        breathingProcess.breathe(); // Operation implemented by
24        Implementor - defines the "HOW"
25    }
26 }
27 public class Tree extends LivingThings {
28     public Tree(BreathingProcess breathingProcess) {
29         super(breathingProcess);
30     }
31
32     @Override
33     public void breathe() {
34        System.out.print("Tree: ");
35        breathingProcess.breathe(); // Operation implemented by
36        Implementor - defines the "HOW"
37    }
38 }

```

Concept && Coding



```

1 // Client Usage
2 public class Client {
3     public static void main(String[] args) {
4         System.out.println("=====  

5         Demo =====");
6         LivingThings dog = new Dog(new LungBreathing());
7         LivingThings fish = new Fish(new GillBreathing());
8         LivingThings tree = new Tree(new Photosynthesis());
9
10        dog.breathe();
11        fish.breathe();
12        tree.breathe();
13    }
14 }

```

The Bridge Pattern allows for the independent addition of new **LivingThings** (e.g., **Frog**) or new Breathing types (e.g., **SkinBreathing** for amphibians). This enables a flexible and extensible design through mixing and matching.

Bridge Pattern vs Strategy Pattern

The patterns are mostly similar but differ in the intent:

- **Bridge**: Focuses on handling two hierarchies independently (abstraction & implementation). Like a **LivingThings** (abstraction) that has different **Respiratory Mechanisms** (implementation). **LivingThings** and **Respiratory Mechanisms** evolve independently.
- **Strategy**: Focuses on handling different behaviors dynamically. Like choosing the driving route on Google Maps, you can switch between "fastest", "shortest", or "avoid tolls" at runtime.

Composite

Definition

The Problem

Solution: Composite Design Pattern

Use cases

Class Diagram

Structure of Composite Pattern

Implementation

Example 1: File Structure

Example 2: Arithmetic Expressions

Resources

- Video → [32. All Structural Design Patterns | Decorator, Proxy, Composite, Adapter, Bridge, Facade, FlyWeight](#)
- Video → [19. Design File System using Composite Design Pattern | Low Level Design Interview Question | LLD](#)

Definition

The Composite Design Pattern allows you to compose objects into tree structures to **represent a part-whole hierarchies** which lets **clients** manage and perform operations on **individual objects(leaf nodes)** and **compositions of objects(composite nodes)** **uniformly** (treating both objects the same way i.e., without knowing whether it's working with a component or a composite).

The Problem

Understanding the problems by implementing the File Structure example using a naive approach:

```
1 public class File {
2     String fileName;
3
4     public File(String name) {
5         this.fileName = name;
6     }
7
8     public void printContents() {
9         System.out.println("File name: " + fileName);
10    }
11 }
```

```
1 public class Directory {
2     String directoryName;
3     List<Object> objectList;
4
5     public Directory(String name) {
6         this.directoryName = name;
7         objectList = new ArrayList<>();
8     }
9
10    public void add(Object object) {
11        objectList.add(object);
12    }
```

```

13
14     public void remove(Object object) {
15         objectList.remove(object);
16     }
17
18     // Display full structure
19     // Breaks OCP - if we want to add a new file type, we need to
    modify this method to add another if/else condition
20     public void printContents() {
21         System.out.println("Directory Name: " + directoryName);
22         for (Object obj: objectList) {
23             if (obj instanceof File) {
24                 ((File) obj).printContents();
25             } else if (obj instanceof Directory) {
26                 ((Directory) obj).printContents();
27             }
28         }
29     }
30 }

```

```

1 // Client Code
2 public class Client {
3     public static void main(String[] args) {
4         Directory movieDirectory = new Directory("Movies");
5
6         File rentalReceipt = new File("RentalReceipt");
7         movieDirectory.add(rentalReceipt);
8
9         Directory comedyMovieDirectory = new
    Directory("ComedyMovies");
10        File dumbAndDumber = new File("DumbAndDumber");
11        comedyMovieDirectory.add(dumbAndDumber);
12        movieDirectory.add(comedyMovieDirectory);
13
14        movieDirectory.printContents();
15    }
16 }

```

Concept & Coding

• No Common Abstraction

- **File** and **Directory** → Both are different types.
- If you write client code, you need to know whether **it is a File or a Folder** before calling methods.

```

public void printContents() {
    System.out.println("Directory Name: " + directoryName);
    for (Object obj : objectList) {
        if (obj instanceof File) {
            ((File) obj).printContents();
        } else if (obj instanceof Directory) {
            ((Directory) obj).printContents();
        }
    }
}

```

- This breaks the Open/Closed Principle. If we want to add a new file type, we need to modify this method to add another **if/else** condition(which is messy).

• Scalability Issue

- If later you add a new type (say a **CompressedFolder**), you have to update everywhere you wrote those **if-else** checks.
- This makes the system rigid, tightly-coupled, and hard to extend.

Solution: Composite Design Pattern

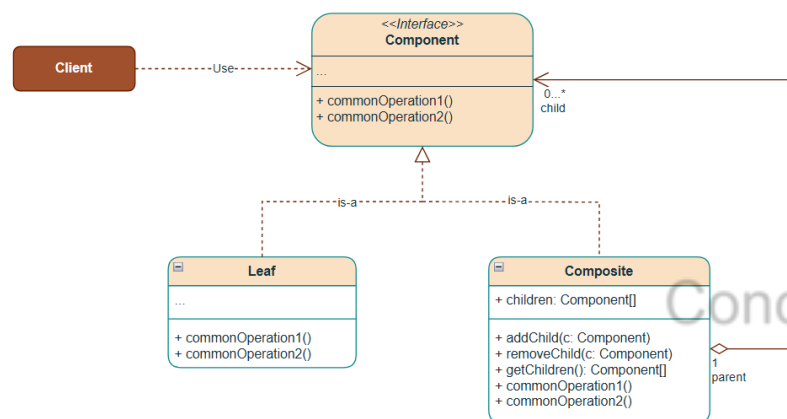
With the Composite Pattern:

- Both **File** and **Folder** implement the same Component interface.
- Client code doesn't have to know whether it's a file or a folder, and hence it can treat both as composite and perform operations uniformly (advantage of using abstraction).
- Adding new types (e.g., **Shortcut** or **ZipFolder**) doesn't require rewriting client logic, making it an extensible design.

Use cases

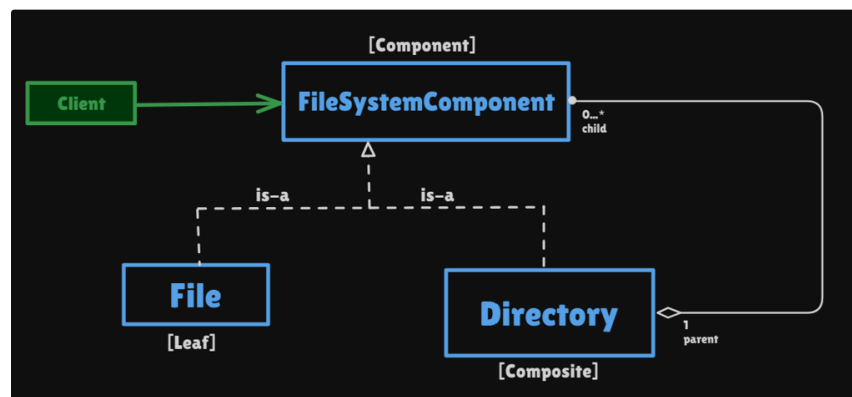
1. File Systems
2. Organizational Structures
3. Mathematical Expressions
4. Building Structures
5. Drawing applications and Computer-Aided Design (CAD)

Class Diagram



Structure of Composite Pattern

Understanding the Structure of Composite Pattern using File Structure Example:



- **Component (FileSystemComponent)**: Abstract base class/Interface defines operations common to both simple (leaf) and complex (composite) objects.
- **Leaf (File)**: Represents a leaf node (end objects) in the composition. A leaf has no children and implements the component interface directly.

- **Composite (Directory)**: Defines the behavior for composites(components that can have children).
- **Client (FileSystemDemo)**: Works with both component and composite objects uniformly through the Component interface.

Implementation

Example 1: File Structure

```
1 // Step 1: Component interface
2 public interface FileSystemComponent {
3     void printContents();
4 }

1 // Step 2: Leaf - File
2 public class File implements FileSystemComponent {
3     String fileName;
4
5     public File(String name) {
6         this.fileName = name;
7     }
8
9     @Override
10    public void printContents() {
11        System.out.println("File name: " + fileName);
12    }
13 }

1 // Step 3: Composite - Folder
2 public class Directory implements FileSystemComponent {
3     String directoryName;
4     List<FileSystemComponent> children;
5
6     public Directory(String name) {
7         this.directoryName = name;
8         children = new ArrayList<>();
9     }
10
11    public void add(FileSystemComponent fileSystemComponent) {
12        children.add(fileSystemComponent);
13    }
14
15    public void remove(FileSystemComponent fileSystemComponent) {
16        children.remove(fileSystemComponent);
17    }
18
19    @Override
20    public void printContents() {
21        System.out.println("Directory Name: " + directoryName);
22        for (FileSystemComponent child : children) {
23            child.printContents();
24        }
25    }
26 }

1 // Step 4: Client code with Composite Pattern
2 public class FileSystemDemo {
3     public static void main(String[] args) {
4         System.out.println("===== Composite Design Pattern =====");
5
6         // Create files
7         File receipt = new File("receipt.pdf");
8         File invoice = new File("invoice.pdf");
9         File torrentLinks = new File("torrentLinks.txt");
10        File tomCruise = new File("tomCruise.jpg");
11        File dumbAndDumber = new File("DumbAndDumber.mp4");
12        File hangoverI = new File("HangoverI.mp4");
```

```

13
14     // Create directories
15     Directory moviesDirectory = new Directory("Movies");
16     Directory comedyMovieDirectory = new
Directory("ComedyMovies");
17
18     // Build the tree structure hierarchically
19     moviesDirectory.add(receipt);
20     moviesDirectory.add(invoice);
21     moviesDirectory.add(torrentLinks);
22     moviesDirectory.add(tomCruise);
23     moviesDirectory.add(comedyMovieDirectory);
24     comedyMovieDirectory.add(dumbAndDumber);
25     comedyMovieDirectory.add(hangoverI);
26
27     // Display full structure
28     moviesDirectory.printContents();
29 }
30 }

```

Example 2: Arithmetic Expressions

```

1 // Component Interface
2 public interface ArithmeticExpression {
3     int evaluate();
4 }

```

```

1 // Leaf
2 public class Number implements ArithmeticExpression {
3     int value;
4
5     public Number(int value) {
6         this.value = value;
7     }
8
9     public int evaluate() {
10        System.out.println("Number value is: " + value);
11        return value;
12    }
13 }

```

```

1 // Composite
2 public class Expression implements ArithmeticExpression {
3
4     ArithmeticExpression leftExpression;
5     ArithmeticExpression rightExpression;
6     OperationType operation;
7
8     public Expression(ArithmeticExpression leftPart,
ArithmeticExpression rightPart, OperationType operation) {
9         this.leftExpression = leftPart;
10        this.rightExpression = rightPart;
11        this.operation = operation;
12    }
13
14    public int evaluate() {
15        int value = 0;
16        switch (operation) {
17            case OperationType.ADD:
18                value = leftExpression.evaluate() +
rightExpression.evaluate();
19                break;
20            case OperationType.SUBTRACT:
21                value = leftExpression.evaluate() -
rightExpression.evaluate();
22                break;
23            case OperationType.DIVIDE:
24                value = leftExpression.evaluate() /
rightExpression.evaluate();
25                break;

```

```

26         case OperationType.MULTIPLY:
27             value = leftExpression.evaluate() *
rightExpression.evaluate();
28             break;
29         }
30
31         System.out.println("Expression value is:" + value);
32         return value;
33     }
34 }

```

```

1 // Client Code with Composite Pattern
2 public class MathExpressionEvaluator {
3     public static void main(String[] args) {
4         System.out.println("==== Composite Design Pattern =====");
5         // 2*(1+7) tree structure for evaluation
6         /*
7
8             *
9         /   \
10      2     +
11           / \
12          1  7
13
14 */
15         ArithmeticExpression two = new Number(2);
16         ArithmeticExpression one = new Number(1);
17         ArithmeticExpression seven = new Number(7);
18
19         ArithmeticExpression addExpression = new Expression(one,
20             seven, OperationType.ADD);
21         ArithmeticExpression parentExpression = new Expression(two,
22             addExpression, OperationType.MULTIPLY);
23
24         System.out.println(parentExpression.evaluate());
25     }
26 }

```

```

1 public enum OperationType {
2     ADD, SUBTRACT, MULTIPLY, DIVIDE
3 }

```

Concept && Coding

Decorator

Definition

Use cases

Example 1: Pizza Shop

Example 2: Coffee Cafe

Why do we need the Decorator Pattern?

Decorator Pattern as a Solution

Class Diagram

Understanding the Structure (e.g., Pizza Shop)

Breakdown of Relationships

Implementation

Code

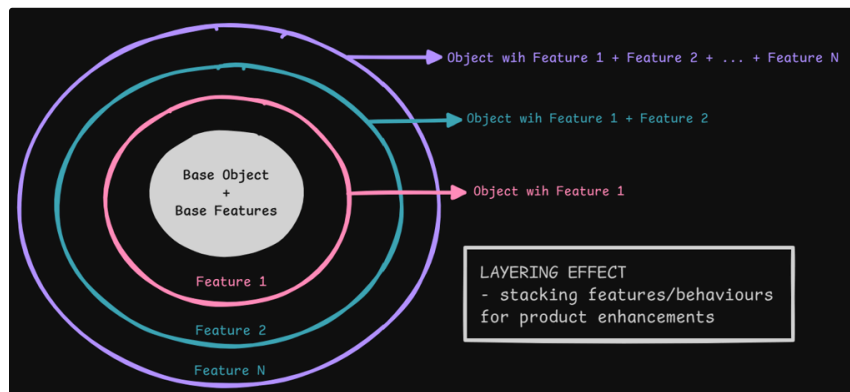
Output

▼ Resources

- Video → [32. All Structural Design Patterns | Decorator, Proxy, Composite, Adapter, Bridge, Facade, FlyWeight](#)
- Video → [4. Decorator Design Pattern Explanation with Java Coding \(Hindi\) | LLD System Design](#)

Definition

Decorator pattern allows you to *add new functionality* to objects *dynamically* without altering their original structure.



Use cases

Example 1: Pizza Shop

• What we have:

- **Basic Pizza:** Margherita (Crust + Cheese)
- **Available Toppings:** Extra Cheese, Olives, Jalapenos, Pepperoni, Veggies, Spicy Red Pepper, etc.

• Pizzas

- **Type 1:** Margherita + Extra Cheese
- **Type 2:** Margherita + Olives + Jalapenos

- **Type 3:** Margherita + Olives + Jalapenos + Veggies + Extra Cheese
- **Type 4:** Margherita + Pepperoni + Spicy Red Pepper

Example 2: Coffee Cafe

• What we have:

- **Espresso:** A strong shot of pure coffee.
- **Available add-ons:** Sugar, Hot water, Cold Water, Ice, Steamed Milk, Milk Foam, Cold Milk, Chocolate Syrup, Vanilla ice cream, etc.

• Coffee Beverages

- **Type 1:** Doppio → A double shot of espresso.
- **Type 2:** Americano → Espresso + Hot water.
- **Type 3:** Cappuccino → Espresso + Steamed Milk + Milk Foam.
- **Type 4:** Mocha → Espresso + Steamed Milk + Chocolate Syrup.
- **Type 5:** Cold Coffee → Espresso + Cold Water + Cold Milk + Ice.
- **Type 6:** MakeYourOwnCoffee → Espresso + Cold Water + Cold Milk + Ice + Vanilla Ice Cream + Chocolate Syrup.

Why do we need the Decorator Pattern?

To grasp the necessity of the Decorator Pattern, we must first look into the challenges encountered when implementing the problem using a naive approach, i.e., without utilizing the Decorator Pattern.

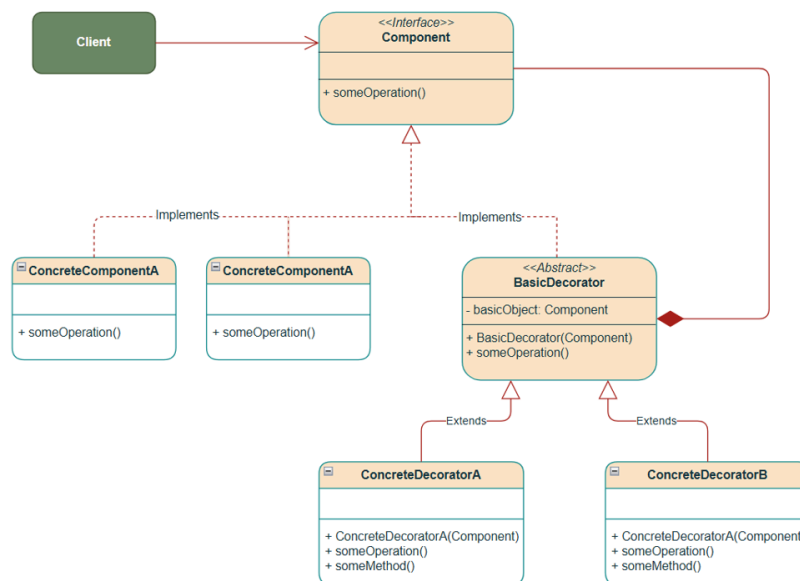
Issue: Class Explosion

- Class explosion refers to a situation where the number of classes in your system grows rapidly and uncontrollably, making the codebase hard to navigate, maintain, and understand.
- This happens when **every combination of additional behavior requires a new decorator class**(if implemented in a naive way).
- You have a base component in the Coffee Cafe example, i.e., `Coffee`, and which has decorators like:
 - `MilkDecorator`, `SugarDecorator`, `VanillaIceCreamDecorator`, `ChocolateSyrupDecorator`, etc.
 - To support all combinations of these decorators, we need to either manually chain them(which doesn't allow changes at runtime) or create **new classes for every common combination**, e.g., `MilkAndSugarCoffeeDecorator`, `SugarAndVanillaDecorator`, `MilkVanillaIceCreamDecorator`, `ChocolateSyrupVanillaIceCreamDecorator`, etc.
 - That's where **class explosion** kicks in; we begin to have **too many classes(extreme subclassing)** to represent all possible combinations of features/behaviours.

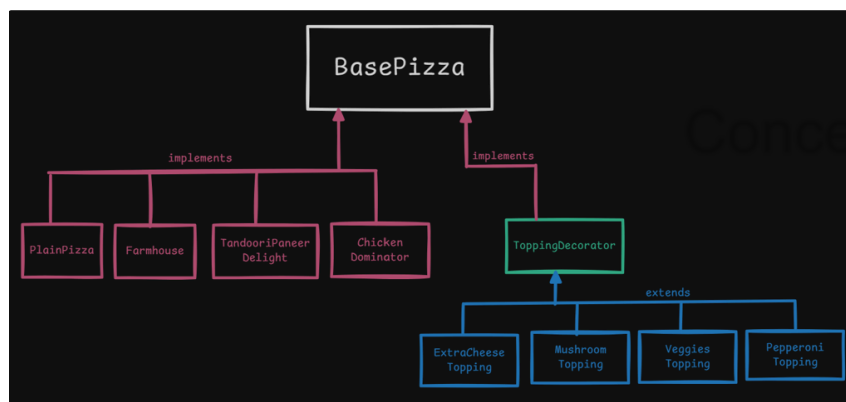
Concept & Coding

Decorator Pattern as a Solution

Class Diagram



Understanding the Structure (e.g., Pizza Shop)



1. Component Interface (BasePizza)

- a. Defines the contract that both the base object and the decorators must follow.
- b. Contains methods like `getDescription()` and `getCost()`.

2. Concrete Component (PlainPizza , Farmhouse , TandooriPaneerDelight , ChickenDominator)

- o The base implementation that provides core functionality.
- o **PlainPizza** represents a simple pizza without any toppings.
- o Can contain a pre-formulated combination of features like **Farmhouse**, **TandooriPaneerDelight**, and **ChickenDominator**.

3. Base Decorator (ToppingDecorator)

- a. An abstract class that implements the Pizza interface.
- b. Contains a reference to a Pizza object - the component itself (composition).

c. Delegates call the wrapped pizza object.

4. **Concrete Decorators** (`ExtraCheeseTopping` , `VeggiesTopping` , `MushroomTopping` , `PepperoniTopping` , etc.)

- Extend the base decorator to add specific functionality.
- Override methods to add their behavior while calling the wrapped object.
- Can modify both the description and cost.

Breakdown of Relationships

1. **IS-A Relationship** (Inheritance-based)

- In the above example, **Concrete Components** (`PlainPizza` , `Farmhouse` , `TandooriPaneerDelight` , `ChickenDominator`) all implement the **Component Interface** (`BasePizza`).
- Hence, they are types of Pizza → **IS-A** Relationship.
- That means
 - `PlainPizza` **is-a** `BasePizza` .
 - `ExtraCheeseTopping` **is-a** `ToppingDecorator` .
 - `ToppingDecorator` **is-a** `BasePizza` .
 - Therefore, `ExtraCheeseTopping` **is-a** `BasePizza` .

2. **HAS-A Relationship** (Composition-based)

- In the above example, `ToppingDecorator` **has a** `BasePizza` .
- `ExtraCheeseTopping` has a `PlainPizza` , which is a `BasePizza` (e.g., Plain Pizza with extra cheese).
- `MushroomTopping` has a `PlainPizza` (e.g., Mushroom Pizza).
- Each new decorator wraps around another `Pizza` , adding its behavior (like toppings).
- That means
 - `ToppingDecorator` **has-a** `Pizza` .
 - Decorators like `ExtraCheeseTopping` , `MushroomTopping` , etc., **have a** reference to another `Pizza` (result of another decorated component or the component itself).

Implementation

Code

```
1 // Step 1: Define the Component Interface
2 public interface BasePizza {
3     String getDescription();
4     double getCost();
5 }
```

```
1 // Step 2: Define the Concrete Component
2 class PlainPizza implements BasePizza {
3     @Override
4     public String getDescription() {
5         return "Plain Pizza";
6     }
}
```

```

7      @Override
8      public double getCost() {
9          return 200.00;
10     }
11 }
12 public class Farmhouse implements BasePizza {
13     @Override
14     public String getDescription() {
15         return "Farmhouse Pizza";
16     }
17     @Override
18     public double getCost() {
19         return 300.0;
20     }
21 }
22 public class TandooriPaneerDelight implements BasePizza {
23     @Override
24     public String getDescription() {
25         return "Tandoori Paneer Delight Pizza";
26     }
27     @Override
28     public double getCost() {
29         return 400.0;
30     }
31 }
32 public class ChickenDominator implements BasePizza {
33     @Override
34     public String getDescription() {
35         return "Chicken Dominator Pizza";
36     }
37     @Override
38     public double getCost() {
39         return 500.0;
40     }
41 }
42 }

```

Concept && Coding

```

1 // Step 3: Define the Abstract Base Decorator
2 public abstract class ToppingDecorator implements BasePizza {
3     BasePizza pizza;
4
5     public ToppingDecorator(BasePizza pizza) {
6         this.pizza = pizza;
7     }
8 }

```

```

1 // Step 4: Define the Concrete Decorator
2 public class ExtraCheeseTopping extends ToppingDecorator {
3
4     public ExtraCheeseTopping(BasePizza pizza) {
5         super(pizza);
6     }
7
8     @Override
9     public String getDescription() {
10         return pizza.getDescription() + " + Extra Cheese";
11     }
12
13     @Override
14     public double getCost() {
15         return pizza.getCost() + 20;
16     }
17 }
18 public class VeggiesTopping extends ToppingDecorator {
19
20     public VeggiesTopping(BasePizza pizza) {
21         super(pizza);
22     }
23
24     @Override
25     public String getDescription() {

```

```

26         return pizza.getDescription() + " + Veggies";
27     }
28
29     @Override
30     public double getCost() {
31         return pizza.getCost() + 30;
32     }
33 }
34 public class MushroomTopping extends ToppingDecorator {
35
36     public MushroomTopping(BasePizza pizza) {
37         super(pizza);
38     }
39
40     @Override
41     public String getDescription() {
42         return pizza.getDescription() + " + Mushroom";
43     }
44
45     @Override
46     public double getCost() {
47         return pizza.getCost() + 40;
48     }
49 }
50 public class PepperoniTopping extends ToppingDecorator {
51
52     public PepperoniTopping(BasePizza pizza) {
53         super(pizza);
54     }
55
56     @Override
57     public String getDescription() {
58         return pizza.getDescription() + " + Pepperoni";
59     }
60
61     @Override
62     public double getCost() {
63         return pizza.getCost() + 50;
64     }
65 }

```

Concept & Coding

```

1 // Step 5: Client Demonstration
2 public class PizzaShop {
3     public static void main(String[] args) {
4         System.out.println("==== Decorator Design Pattern =====");
5         // Create a plain pizza
6         BasePizza pizza1 = new PlainPizza();
7         System.out.println("Order 1: " + pizza1.getDescription() + " =
Rs." + pizza1.getCost());
8
9         // Add toppings to the PlainPizza - Extra Cheese Only
10        BasePizza pizza2 = new ExtraCheeseTopping(new PlainPizza());
11        System.out.println("Order 2: " + pizza2.getDescription() + " =
Rs." + pizza2.getCost());
12
13        // Add toppings to the PlainPizza - Extra Cheese and Veggies
14        BasePizza pizza3 = new VeggiesTopping(new
ExtraCheeseTopping(new PlainPizza()));
15        System.out.println("Order 3: " + pizza3.getDescription() + " =
Rs." + pizza3.getCost());
16
17        // Add toppings to the PlainPizza - Extra Cheese and Pepperoni
18        BasePizza pizza4 = new PepperoniTopping(new
ExtraCheeseTopping(new PlainPizza()));
19        System.out.println("Order 4: " + pizza4.getDescription() + " =
Rs." + pizza4.getCost());
20
21        // Add toppings to the PlainPizza - Extra Cheese, Mushroom and
Pepperoni
22        BasePizza pizza5 = new MushroomTopping(new
PepperoniTopping(new ExtraCheeseTopping(new PlainPizza())));

```

```

23         System.out.println("Order 5: " + pizza5.getDescription() + " =
Rs." + pizza5.getCost());
24
25         // Farmhouse Pizza
26         BasePizza pizza6 = new Farmhouse();
27         System.out.println("Order 6: " + pizza6.getDescription() + " =
Rs." + pizza6.getCost());
28
29         // Farmhouse Pizza with Extra Cheese and Mushroom
30         BasePizza pizza7 = new MushroomTopping(new
ExtraCheeseTopping(new Farmhouse()));
31         System.out.println("Order 7: " + pizza7.getDescription() + " =
Rs." + pizza7.getCost());
32
33         // Tandoori Paneer Delight Pizza
34         BasePizza pizza8 = new TandooriPaneerDelight();
35         System.out.println("Order 8: " + pizza8.getDescription() + " =
Rs." + pizza8.getCost());
36
37         // Chicken Dominator
38         BasePizza pizza9 = new ChickenDominator();
39         System.out.println("Order 9: " + pizza9.getDescription() + " =
Rs." + pizza9.getCost());
40
41         // Chicken Dominator with Mushroom
42         BasePizza pizza10 = new MushroomTopping(new
ChickenDominator());
43         System.out.println("Order 10: " + pizza10.getDescription() + "
= Rs." + pizza10.getCost());
44     }
45 }

```

Output

```

===== Decorator Design Pattern =====
Order 1: Plain Pizza = Rs.200.0
Order 2: Plain Pizza + Extra Cheese = Rs.220.0
Order 3: Plain Pizza + Extra Cheese + Veggies = Rs.250.0
Order 4: Plain Pizza + Extra Cheese + Pepperoni = Rs.270.0
Order 5: Plain Pizza + Extra Cheese + Pepperoni + Mushroom = Rs.310.0
Order 6: Farmhouse Pizza = Rs.300.0
Order 7: Farmhouse Pizza + Extra Cheese + Mushroom = Rs.360.0
Order 8: Tandoori Paneer Delight Pizza = Rs.400.0
Order 9: Chicken Dominator Pizza = Rs.500.0
Order 10: Chicken Dominator Pizza + Mushroom = Rs.540.0

Process finished with exit code 0

```

The key benefit of the Decorator pattern is that it provides a **flexible alternative** to subclassing for extending functionality, allowing you to **mix and match behaviors as needed at runtime**.

Facade

Definition

Facade Pattern Use-cases

Real Life Example: Car

Scenario 1: Employee Operations

Scenario 2: Order Processing

The Problem(Without the Facade Pattern)

Scenario 3: Facade Using another Facade

Class Diagram

Structure Of Facade Pattern

Implementation

Facade vs Proxy Pattern

Facade vs Adapter Pattern

▼ Resources

- Video → [32. All Structural Design Patterns | Decorator, Proxy, Composite, Adapter, Bridge, Facade, FlyWeight](#)
- Video → [25. Facade Design Pattern with Example | Facade Low Level Design Pattern | Facade Pattern LLD Java](#)

Definition

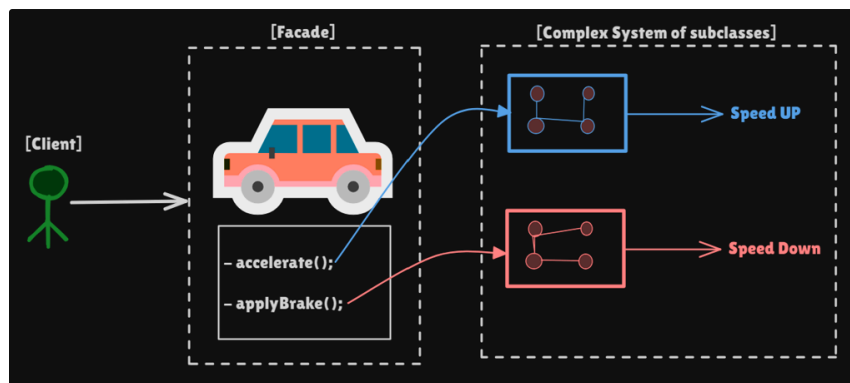
*The Facade Design Pattern provides a **simplified interface** to a **complex subsystem** for the **client** to interact with it seamlessly.*

Facade Pattern Use-cases

✓ When and why do we use the Facade Design Pattern in the code?

Whenever we have to **hide system complexity** from the client, we use the Facade Design Pattern.

Real Life Example: Car

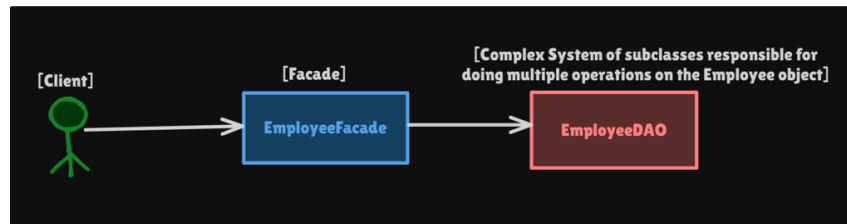


Think of it like using `accelerator()` and `brake()` features in a `Car`. The `Car` conceals the intricate complexities of smaller hardware processes (subsystems), which are interconnected to sequentially manage the actions (speeding or slowing down) once the driver presses the pedals while driving.

- It is also important to note that the **client doesn't need** to use a facade to communicate with the subsystem components. It can always choose to communicate with the components and carry out the execution by itself.

Scenario 1: Employee Operations

Expose only a few methods/features the client is interested in using.

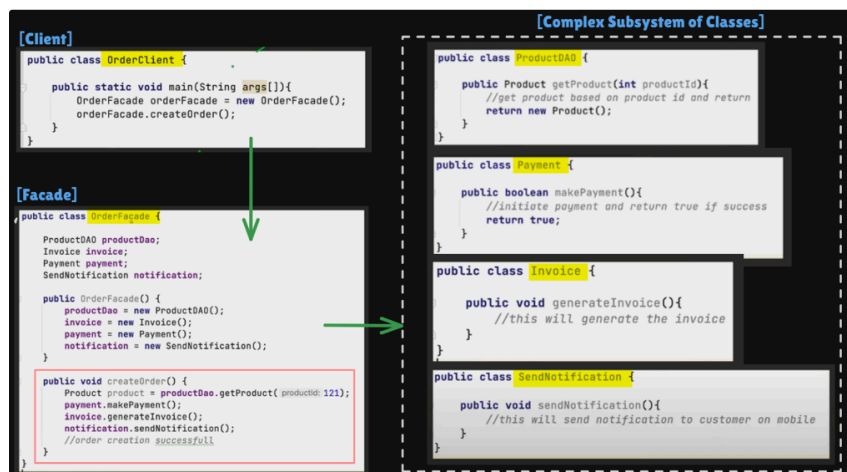
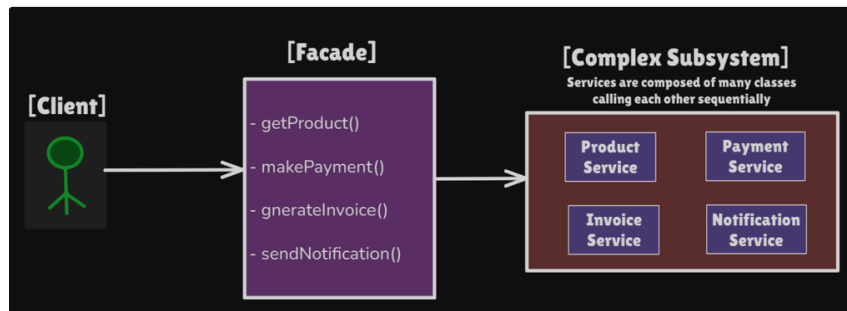


```
public class EmployeeClient {  
    public void getEmployeeDetails() {  
        EmployeeFacade employeeFacade = new EmployeeFacade();  
        Employee employeeDetails = employeeFacade.getEmployeeDetails( empID: 121222);  
    }  
}
```

```
public class EmployeeFacade {  
    EmployeeDAO employeeDAO;  
    public EmployeeFacade(){  
        employeeDAO = new EmployeeDAO();  
    }  
    public void insert(){  
        employeeDAO.insert();  
    }  
    public Employee getEmployeeDetails(int empID){  
        return employeeDAO.getEmployeeDetails(empID);  
    }  
}
```

```
public class EmployeeDAO {  
    public void insert(){  
        //insert into Employee Table  
    }  
    public void updateEmployeeName(){  
        //updating employee Name  
    }  
    public Employee getEmployeeDetails(String emailID){  
        //get employee details based on Emp ID  
        return new Employee();  
    }  
    public Employee getEmployeeDetails(int empID){  
        //get employee details based on Emp ID  
        return new Employee();  
    }  
}
```

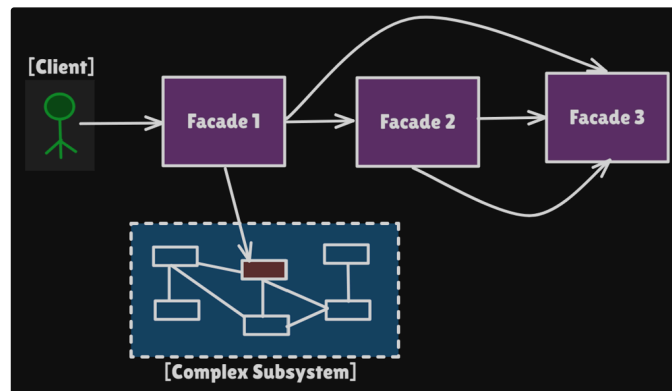
Scenario 2: Order Processing



The Problem(Without the Facade Pattern)

- The client has to talk to every subsystem directly and implement the proper sequence of execution order.
- No data hiding or encapsulation. The client has too much exposure to subsystems, making it tightly coupled with the internal structure.
- Hard to change/introduce new steps. If the order process changes (e.g., new `DiscountService` or different payment flow), you must update every client that uses it.
- It would be more error-prone, e.g., the Client might forget a step (e.g., `sendConfirmation()`).
Or call it in the wrong order (payment before stock availability check).

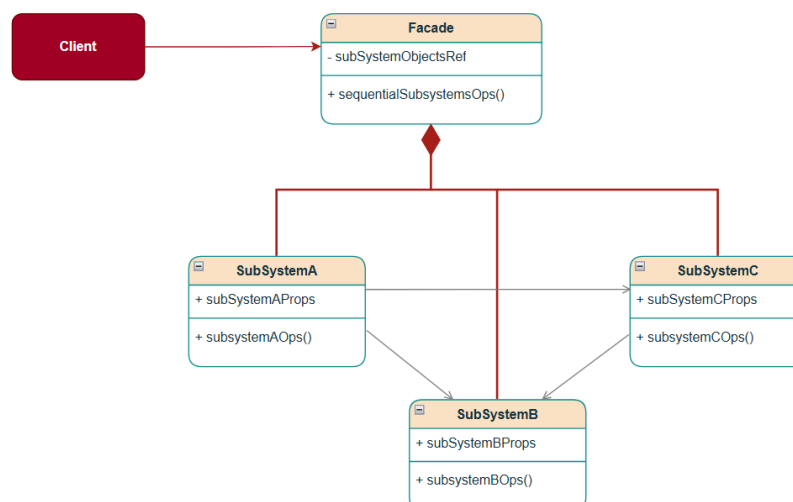
Scenario 3: Facade Using another Facade



A facade can use another facade or a subsystem directly, depending on the requirement and complexity. The facade pattern provides a single entry point to the client for interaction.

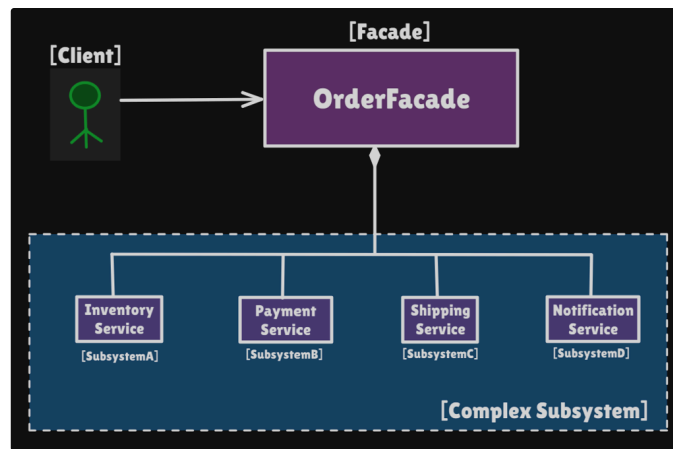
Concept & Coding

Class Diagram



Structure Of Facade Pattern

Let's look at the Structure of the Facade Pattern using the Order Processing example as discussed above.



- **Facade (OrderFacade)**: Holds references to all downstream subsystems(that are working together to complete an action) and defines a method to perform the required operation, i.e., `OrderFacade.placeOrder()` .
- **SubsystemA (InventoryService)**: This service checks if items are in stock.
- **SubsystemB (PaymentService)**: This service processes payments.
- **SubsystemC (ShippingService)**: This service arranges delivery.
- **SubsystemD (NotificationService)**: This service sends confirmation.
- **Client(ECommerceApp)**: Client places an order with just one call to the facade class `OrderFacade.placeOrder()` .

Concept & Coding

Implementation

Let's look at the Implementation of the Facade Pattern using the **Order Processing example** as discussed above.

```

1  // Complex Subsystems
2
3  // Subsystem A: Inventory
4  public class InventoryService {
5      public boolean checkStock(String productId) {
6          System.out.println("Checking stock for product: " +
7              productId);
8          return true; // assume it's always in stock
9      }
10 }
11 // Subsystem B: Payment
12 public class PaymentService {
13     public boolean makePayment(String paymentMethod) {
14         System.out.println("Processing payment using: " +
15             paymentMethod);
16         return true; // assume payment always succeeds
17     }
18 }
19 // Subsystem C: Shipping
20 public class ShippingService {
21     public void shipProduct(String productId) {
22         System.out.println("Shipping product: " + productId);
23     }
24 }
25 // Subsystem D: Notification
26 public class NotificationService {
27     public void sendConfirmation(String productId) {
28         System.out.println("Sending order confirmation for product: "
29             + productId);
30     }
31 }
  
```

```
28 }
```

```
1 // Facade hides complexity and provides a simple unified interface
2 public class OrderFacade {
3     private final InventoryService inventory;
4     private final PaymentService payment;
5     private final ShippingService shipping;
6     private final NotificationService notification;
7
8     public OrderFacade() {
9         this.inventory = new InventoryService();
10        this.payment = new PaymentService();
11        this.shipping = new ShippingService();
12        this.notification = new NotificationService();
13    }
14
15    // Simplified method for clients
16    public void placeOrder(String productId, String paymentMethod) {
17
18        // The following steps are hidden from the client and need to
19        // be executed in a specific order
20        System.out.println("Placing order for product: " + productId);
21
22        // Step 1: Check stock
23        if (!inventory.checkStock(productId)) {
24            System.out.println("Product out of stock!");
25            return;
26        }
27
28        // Step 2: Make payment
29        if (!payment.makePayment(paymentMethod)) {
30            System.out.println("Payment failed!");
31            return;
32        }
33
34        // Step 3: Ship product
35        shipping.shipProduct(productId);
36
37        // Step 4: Send confirmation
38        notification.sendConfirmation(productId);
39
40        System.out.println("Order placed successfully!");
41    }
42 }
```

```
1 // Client Usage
2 public class ECommerceApp {
3     public static void main(String[] args) {
4         System.out.println("==== Facade Design Pattern Demo
5         =====");
6         // Client interacts with a simple Facade, not with all
7         // subsystems.
8         OrderFacade orderFacade = new OrderFacade();
9
10        // Place order with one call to Facade
11        orderFacade.placeOrder("MacBook Pro", "Credit Card");
12
13        // Place another order with one call to Facade
14        orderFacade.placeOrder("Cricket Bat", "UPI");
15    }
16 }
```

Concept && Coding

Facade vs Proxy Pattern

Facade and Proxy are both structural patterns, but they solve different problems.

Intent

- **Facade Pattern:** Provides a simplified interface to a complex subsystem to facilitate clients' interaction. The main purpose is to reduce complexity for clients. It is associated with references to many objects involved in the specific task.
- **Proxy Pattern:** Acts as a surrogate/placeholder/substitute for another object to control access to it. It is used to add more control (security, logging, caching, lazy loading) and is associated with only ONE object directly.

Facade vs Adapter Pattern

Facade and Adapter often look alike because both give a different interface to clients, but differ in the following:

Intent

- **Facade Pattern:** Provides a simplified interface to a complex subsystem to facilitate clients' interaction and hide the complexity.
- **Adapter Pattern:** Provides a way to use an existing incompatible interface that the client expects. Improves compatibility, making two interfaces work together.

Concept && Coding

Flyweight

Definition

When do we use this?

Class Diagram

Structure of the Flyweight Pattern

Example 1: Robotic Game

Robotic Game: Issue

Robotic Game: Flyweight Implementation as a Solution

Example 2: Word Processor

Word Processor: Issue

Word Processor: Flyweight Implementation as a Solution

▼ Resources

• Video → [32. All Structural Design Patterns | Decorator, Proxy, Composite, Adapter, Bridge, Facade, FlyWeight](#)

• Video → [30. Design Word Processor using Flyweight Design Pattern | Low Level System Design FlyWeight Pattern](#)

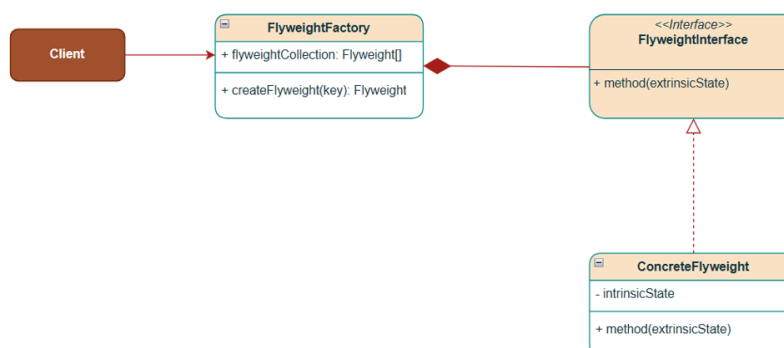
Definition

*The Flyweight Design Pattern is a structural pattern that helps **reduce memory usage** by efficiently **sharing data** that is common to multiple similar objects. This pattern is widely used in applications where it is required to **generate a large number of similar objects**.*

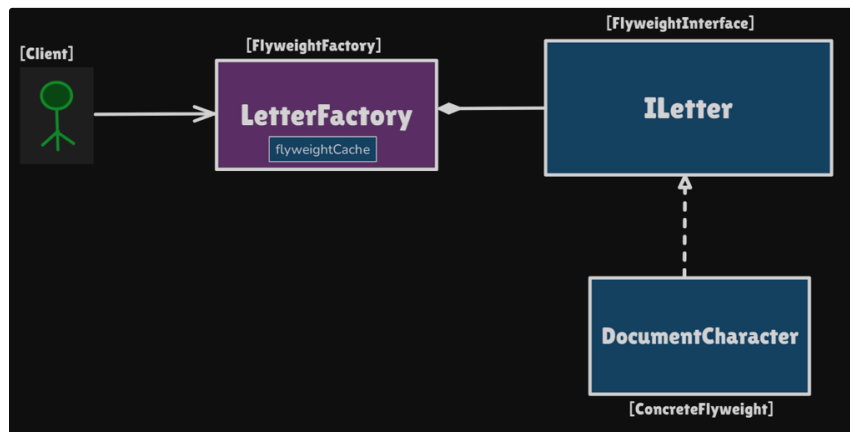
When do we use this?

- When Memory is Limited.
- When Objects share data.
 - **Intrinsic data:** Data that is shared among objects and remains the same once it is set.
 - **Extrinsic data:** This data changes based on client input and differs from one object to another.
- Creation of an Object is expensive.

Class Diagram



Structure of the Flyweight Pattern



Refer to the section below for code:

[Flyweight | Word Processor: Flyweight Implementation as a Solution](#)

- **Flyweight Interface(ILetter)**: Defines methods that use extrinsic state.
- **ConcreteFlyweight(DocumentCharacter)**: Stores intrinsic (shared) state.
- **FlyweightFactory(LetterFactory)**: Manages & reuses flyweights.
- **Client(WordProcessorSimulation)**: Supplies extrinsic state when using flyweights.

Example 1: Robotic Game

Robotic Game: Issue

Concept & Coding

Let's look at the naive approach in creating a game where we usually tend to create a lot of similar objects.

```
1 // Sprites class is a heavy-weight object
2 public class Sprites {
3     // In computer graphics, a sprite is a two-dimensional bitmap image
    used to
4     // represent a character, object, or other element in a video game
    or other
5     // computer-generated image.
6 }
```

```
1 // Robot class - Used to create Humanoid and Robotic Dog robots
2 public class Robot {
3     int coordinateX;
4     int coordinateY;
5     String type;
6     Sprites body; // heavy-weight object - 2D bitmap image
7
8     Robot(int coordinateX, int coordinateY, String type, Sprites body)
9     {
10         this.coordinateX = coordinateX;
11         this.coordinateY = coordinateY;
12         this.type = type;
13         this.body = body;
14     }
15     // getters and setters
16 }
```

```
1 // Client Code - Robotic game creating robots
2 public class Main {
3     public static void main(String[] args) {
4         int x = 0;
5         int y = 0;
```

```

6         // Create 5L Humanoid robots
7         for (int i = 0; i < 500000; i++) {
8             Sprites humanoidSprite = new Sprites();
9             Robot humanoidRobotObject = new Robot(x + i, y + i,
10            "HUMANOID", humanoidSprite);
11        }
12        // Create 5L Robotic Dog robots
13        for (int i = 0; i < 500000; i++) {
14            Sprites roboticDogSprite = new Sprites();
15            Robot roboticDogObject = new Robot(x + i, y + i,
16            "ROBOTIC_DOGS", roboticDogSprite);
17        }
18        // A total of 10L robots created will result in 10L Sprite
19        objects created
20        // which will consume a lot of memory.
21    }
22 }

```

A total of 10 lakh robots created will result in 10 lakh Sprite objects created, which will consume a lot of memory (let's say each robot is 40 KB, $40 \text{ kilobytes} \times 10 \text{ lakh} = 40 \text{ gigabytes}$), and if it exceeds the system's capacity (32 GB RAM), the application might become unresponsive or most probably crash leading to bad user experience and unavailability.

Robotic Game: Flyweight Implementation as a Solution

Let's see how we can solve the issue:

- From the above **Robot** object, remove all the extrinsic data and keep the intrinsic data; it will result in a *flyweight object*.
- This Flyweight Class can be immutable. Provide getter methods only.
- Extrinsic Data can be passed to the Flyweight class as a method parameter.
- Once the Flyweight Object is created, it is **cached** and **reused** whenever required.

Concept && Coding

```

1 // Sprites class is a heavyweight object
2 public class Sprites {
3     // In computer graphics, a sprite is a two-dimensional bitmap
4     // image used to represent a character, object, or other element in a video game
5     // or other computer-generated image.
6 }
7 // Flyweight (Interface) - for the flyweight object - defines methods
8 // that use extrinsic state.
9 public interface IRobot {
10     // CoordinateX and CoordinateY are extrinsic data - unique to each
11     // object
12     void display(int x, int y);
13 }

```

```

1 // Concrete Flyweight (Class) - implements the Flyweight interface and
2 // stores intrinsic state.
3 public class HumanoidRobot implements IRobot {
4     // intrinsic data - shared data - common to all objects
5     private final String type; // humanoid or robotic dog
6     private final Sprites body; //small 2d bitmap (graphic element)
7
8     HumanoidRobot(String type, Sprites body) {
9         this.type = type;
10        this.body = body;
11    }
12
13    public String getType() {
14        return type;
15    }
16 }

```

```

15
16     public Sprites getBody() {
17         return body;
18     }
19
20     @Override
21     public void display(int x, int y) {
22         // use the humanoid sprites object
23         // and X and Y coordinate to render the image.
24         System.out.println("Displaying " + type + " at " + x + ", " +
25         y);
26     }
27 }

```

```

1 // Concrete Flyweight (Class) - implements the Flyweight interface and
2 // stores intrinsic state.
3 public class RoboticDog implements IRobot {
4     // intrinsic data - shared data - common to all objects
5     private final String type; // humanoid or robotic dog
6     private final Sprites body; // small 2d bitmap (graphic element)
7
8     RoboticDog(String type, Sprites body) {
9         this.type = type;
10        this.body = body;
11    }
12
13    public String getType() {
14        return type;
15    }
16
17    public Sprites getBody() {
18        return body;
19    }
20
21    @Override
22    public void display(int x, int y) {
23        //use the Robotic Dog sprites object
24        // and X and Y coordinate to render the image.
25        System.out.println("Displaying " + type + " at " + x + ", " +
26        y);
27    }
28 }

```

```

1 // Flyweight Factory (Class) - creates and manages flyweight objects.
2 public class RoboticFactory {
3
4     private static final Map<String, IRobot> roboticObjectCache = new
5     HashMap<>();
6
7     public static IRobot createRobot(String robotType) {
8         if (roboticObjectCache.containsKey(robotType)) {
9             // if exists, return the cached object.
10            return roboticObjectCache.get(robotType);
11        } else {
12            // if not exists, create the object and cache it.
13            if (robotType.equals("HUMANOID")) {
14                Sprites humanoidSprite = new Sprites();
15                IRobot humanoidObject = new HumanoidRobot(robotType,
16                humanoidSprite);
17                // add to cache
18                roboticObjectCache.put(robotType, humanoidObject);
19                return humanoidObject;
20            } else if (robotType.equals("ROBOTIC_DOG")) {
21                Sprites roboticDogSprite = new Sprites();
22                IRobot roboticDogObject = new RoboticDog(robotType,
23                roboticDogSprite);
24                // add to cache
25                roboticObjectCache.put(robotType, roboticDogObject);
26                return roboticDogObject;
27            }
28        }
29    }
30 }

```



```

26         throw new IllegalArgumentException("Invalid robot type: " +
robotType);
27     }
28
29     public static int getTotalRobots() {
30         return roboticObjectCache.size();
31     }
32 }

```

```

1  // Client - supplies extrinsic state when using flyweights.
2  public class RoboticGameSimulation {
3      public static void main(String[] args) {
4          System.out.println("==== Flyweight Design Pattern =====");
5          // Factory pattern is used to create objects
6          // Flyweight pattern is used to reuse objects
7
8          // Create 2 Humanoid robots and provide display
coordinates(extrinsic state) at runtime
9          IRobot humanoidRobot1 =
RoboticFactory.createRobot("HUMANOID");
10         humanoidRobot1.display(1, 2);
11         IRobot humanoidRobot2 =
RoboticFactory.createRobot("HUMANOID");
12         humanoidRobot2.display(10, 30);
13
14         // Create 2 Robotic Dog robots and provide display
coordinates(extrinsic state) at runtime
15         IRobot roboDog1 = RoboticFactory.createRobot("ROBOTIC_DOG");
16         roboDog1.display(2, 9);
17         IRobot roboDog2 = RoboticFactory.createRobot("ROBOTIC_DOG");
18         roboDog2.display(11, 19);
19
20         // Total robots created: 2 - because we are reusing the same
object
21         System.out.println("Total robots created: " +
RoboticFactory.getTotalRobots());
22     }
23 }

```

Concept & Coding

Here, we created a total of 4 objects(2 humanoids and 2 robotic dogs), but with the flyweight pattern approach, we are consuming 50% less memory(only 2 objects created) by reusing the objects.

Example 2: Word Processor

Word Processor: Issue

```

1  public class Character {
2      char character;
3      String fontType;
4      int size;
5      int row;
6      int column;
7
8      public Character(char character, String fontType, int size, int
row, int column) {
9          this.character = character;
10         this.fontType = fontType;
11         this.size = size;
12         this.row = row;
13         this.column = column;
14     }
15 }

```

```

1  public class Demo {
2      public static void main(String[] args) {
3          System.out.println("Word Processor: Issue Demo");
4          // Data: "Hello World"
5          // Total 11 characters

```

```

6      // h = 1 time
7      // e = 1 time
8      // l = 3 times
9      // o = 2 times
10     // w = 1 time
11     // r = 1 time
12     // d = 1 time
13     // ' ' = 1 time
14
15     // Create 11 character objects
16     Character object1 = new Character('H', "Arial", 10, 0, 0);
17     Character object2 = new Character('e', "Arial", 10, 0, 1);
18     Character object3 = new Character('l', "Arial", 10, 0, 2);
19     Character object4 = new Character('l', "Arial", 10, 0, 3);
20     Character object5 = new Character('o', "Arial", 10, 0, 4);
21     Character object6 = new Character(' ', "Arial", 10, 0, 5);
22     Character object7 = new Character('W', "Arial", 10, 0, 6);
23     Character object8 = new Character('o', "Arial", 10, 0, 7);
24     Character object9 = new Character('r', "Arial", 10, 0, 8);
25     Character object10 = new Character('l', "Arial", 10, 0, 9);
26     Character object11 = new Character('d', "Arial", 10, 0, 10);
27 }
28 }

```

A real document may have millions of characters. Implementing a word processor using a naive approach, i.e., without flyweight → we would store "A" object 5000 times → memory heavy. This would result in excessive memory usage, causing our application to crash as discussed above.

Word Processor: Flyweight Implementation as a Solution

```

1 // Flyweight (Interface) - for the flyweight object - defines methods
  that use extrinsic state.
2 public interface ILetter {
3     // The position(row,column) is extrinsic data - unique to each
  object
4     void display(int row, int column);
5 }

```

```

1 // Concrete Flyweight (Class) - implements the Flyweight interface and
  stores intrinsic state
2 public class DocumentCharacter implements ILetter {
3     // intrinsic data - shared data - common to all objects
4     private final char character;
5     private final String fontType;
6     private final int size;
7
8     DocumentCharacter(char character, String fontType, int size) {
9         this.character = character;
10        this.fontType = fontType;
11        this.size = size;
12    }
13
14    // getter methods only
15
16    @Override
17    public void display(int row, int column) {
18        //display the character of particular font and size at given
  location
19        System.out.println("Displaying " + character + " at row " +
  row + " and column " + column);
20    }
21
22 }

```

```

1 // Flyweight Factory (Class) - creates and manages flyweight objects.
2 public class LetterFactory {
3
4     private static final Map<Character, ILetter> characterCache = new
  HashMap<>();

```

```

5
6     public static ILetter crateLetter(char characterValue) {
7         if (characterCache.containsKey(characterValue)) {
8             // if exists, return the cached character object.
9             return characterCache.get(characterValue);
10        } else {
11            // if not exists, create the character object and cache
12            it. DocumentCharacter characterObj = new
13            DocumentCharacter(characterValue, "Arial", 10);
14            // add to cache
15            characterCache.put(characterValue, characterObj);
16            return characterObj;
17        }
18
19        public static int getTotalCharacters() {
20            return characterCache.size();
21        }
22    }

```

```

1 // Client - supplies extrinsic state when using flyweights
2 public class WordProcessorSimulation {
3     public static void main(String[] args) {
4         // Data: "Hello World"
5         // Total 11 characters (including space)
6         // h = 1 time
7         // e = 1 time
8         // l = 3 times (reused)
9         // o = 2 times (reused)
10        // w = 1 time
11        // r = 1 time
12        // d = 1 time
13        // ' ' = 1 time
14
15        // Create 11 character objects and provide display
16        position(extrinsic state) at runtime
17        ILetter object1 = LetterFactory.crateLetter('H');
18        object1.display(0, 0);
19
20        ILetter object2 = LetterFactory.crateLetter('e');
21        object2.display(0, 1);
22
23        ILetter object3 = LetterFactory.crateLetter('l');
24        object3.display(0, 2);
25
26        ILetter object4 = LetterFactory.crateLetter('l');
27        object4.display(0, 3);
28
29        ILetter object5 = LetterFactory.crateLetter('o');
30        object5.display(0, 4);
31
32        ILetter object6 = LetterFactory.crateLetter(' ');
33        object6.display(0, 5);
34
35        ILetter object7 = LetterFactory.crateLetter('W');
36        object7.display(0, 6);
37
38        ILetter object8 = LetterFactory.crateLetter('o');
39        object8.display(0, 7);
40
41        ILetter object9 = LetterFactory.crateLetter('r');
42        object9.display(0, 8);
43
44        ILetter object10 = LetterFactory.crateLetter('l');
45        object10.display(0, 9);
46
47        ILetter object11 = LetterFactory.crateLetter('d');
48        object11.display(0, 10);

```

Concept & Coding

```
49      // Total characters created: 8 - because we are reusing the
      same object
50      System.out.println("Total characters created: " +
      LetterFactory.getTotalCharacters());
51  }
52 }
```

A real document may have millions of characters. Using the flyweight pattern, we store "A" just once and reuse it with extrinsic state (position or formatting). This approach proves to be a memory saver, reuses shared data, and passes varying context(extrinsic state) separately.

Concept && Coding

Proxy

Definition

Use cases of the Proxy Pattern

Class Diagram

Structure of Proxy Pattern

Implementation

▼ Resources

- Video → [32. All Structural Design Patterns | Decorator, Proxy, Composite, Adapter, Bridge, Facade, FlyWeight](#)
- Video → [13. Proxy Design Pattern Explanation \(Hindi\) | LLD | System Design Interview Question | Java](#)

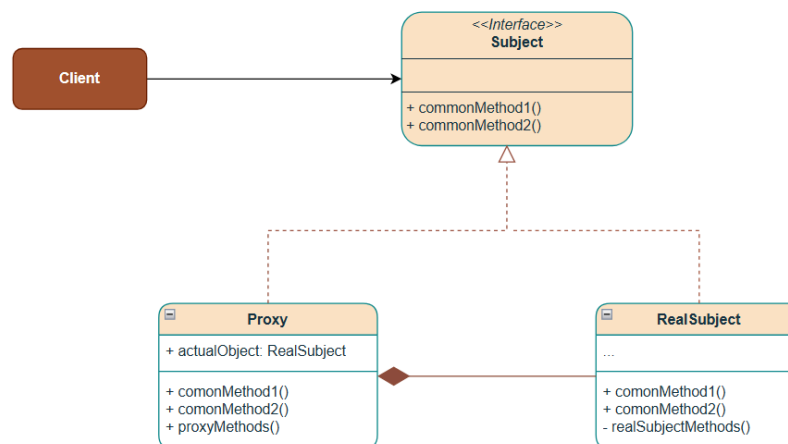
Definition

*The Proxy Design Pattern provides a representative or **placeholder** for another **object** (real complex object) to **control access** to it.*

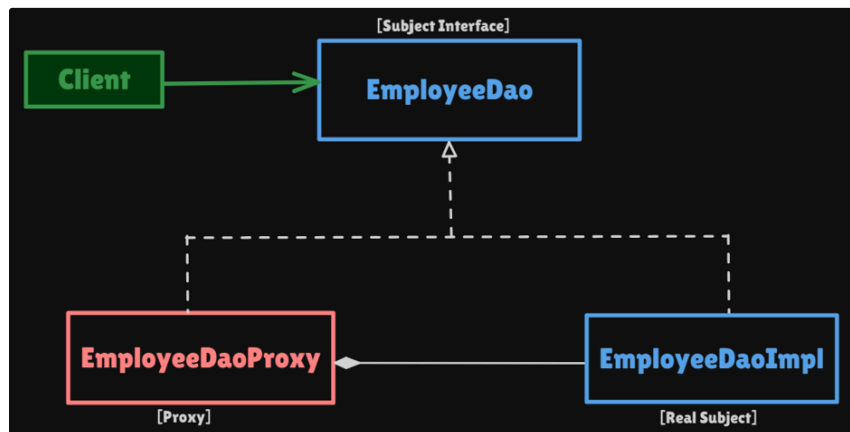
Use cases of the Proxy Pattern

- **Access Control:** Restricts access to sensitive operations based on user permissions.
- **Performance Optimization:** Delays resource-intensive operations like expensive object creation until necessary (lazy loading).
- **Pre/post processing:** Performs additional operations before and after forwarding the request to the actual object, like adding cross-cutting concerns like audit traces (logging & monitoring), without modifying core business logic.
- **Caching:** Can cache results to improve performance.

Class Diagram



Structure of Proxy Pattern



1. Subject Interface (EmployeeDao)

- Defines the common interface for both EmployeeDaoImpl and EmployeeDaoProxy .
- Ensures the proxy can be used anywhere the real object is expected

2. Real Subject (EmployeeDaoImpl)

- Contains the actual business logic and data. Handles core employee operations.
- Performs expensive operations like database loading.

3. Proxy Object (EmployeeDaoProxy)

- Maintains a reference to the Real Subject(EmployeeDaoImpl).
- Controls access through permission checks.
- Can be modified further to implement lazy loading, logging and etc.

4. Client (EmployeeManagement)

- Client is performing operations on the subject without being aware of whether it's a Real/Proxy object

Concept && Coding

Implementation

```
1 public interface EmployeeDao {
2     void getEmployeeInfo(int empID);
3     void createEmployee(EmployeeDo obj);
4 }
```

```
1 public class EmployeeDaoImpl implements EmployeeDao {
2     public void getEmployeeInfo(int empID) {
3         System.out.println("Fetching employee info for ID: " + empID);
4     }
5
6     public void createEmployee(EmployeeDo obj) {
7         System.out.println("Creating employee: " + obj);
8     }
9 }
```

```
1 public class EmployeeDaoProxy implements EmployeeDao {
2
3     private EmployeeDao empDaoObj;
4     private String clientRole;
5
6     public EmployeeDaoProxy(String clientRole) {
7         empDaoObj = new EmployeeDaoImpl();
8         this.clientRole = clientRole;
9     }
10 }
```

```

11     @Override
12     public void getEmployeeInfo(int empID) {
13         if (clientRole.equals("ADMIN") || clientRole.equals("USER")) {
14             empDaoObj.getEmployeeInfo(empID);
15         } else {
16             throw new RuntimeException("Access Denied");
17         }
18     }
19
20     @Override
21     public void createEmployee(EmployeeDo obj) {
22         if (clientRole.equals("ADMIN")) {
23             empDaoObj.createEmployee(obj);
24         } else {
25             throw new RuntimeException("Access Denied");
26         }
27     }
28 }

```

```

1 // Client
2 public class EmployeeManagement {
3     public static void main(String[] args) {
4         System.out.println("==== Proxy Design Pattern =====");
5
6
7         EmployeeDao userProxyObj = new EmployeeDaoProxy("USER");
8         userProxyObj.getEmployeeInfo(); //access granted
9         userProxyObj.createEmployee(new EmployeeDo()); //access denied
10    }
11 }

```

The Proxy pattern is particularly useful in enterprise applications where security, performance, and monitoring are critical concerns.

Concept && Coding